using Tailwind CSS in your Rails app, there's another approach to consider.

Tailwind CSS

For those who are unfamiliar with Tailwind CSS, here's a bit of background:

Tailwind CSS[23] is a utility-first CSS framework. This means you won't find semantic classes like **card** or even **container** here. Instead, it uses a plethora of tiny ones like **flex**, **pt-4**, **text-center**, and **rotate-90**.

The benefit of using Tailwind is that you can use these utility classes to compose *any* design, directly in your markup, without having to also juggle and maintain CSS by hand.

Where Bootstrap has responsive breakpoints, Tailwind CSS has *variants*. These handle not only screen sizes but also hover state, color scheme, and more. Variants are declared with the variant name, followed by a colon, and then the class name to apply.

Here are some examples:

```
<div class="bg-white md:bg-black">
  White background on small screens, black on medium and larger.
</div>

<div class="text-black dark:text-white">
  Black text by default, white text when dark mode is enabled.
</div>
```

With Tailwind CSS, you can create a custom modifier that only applies to Hotwire Native apps. Normally, we identify these requests by inspecting the user agent. But Tailwind CSS modifiers are written in CSS, which can't access the request or the user agent.

The Rails app in this book uses Bootstrap. But if your app is using Tailwind CSS, you can identify the requests by adding the **data-hotwire-native-app** attribute to the **<html>** tag in your application layout:

ch02_10/rails/app/views/layouts/application.html.erb